

# Queue - 1

# In This Lecture

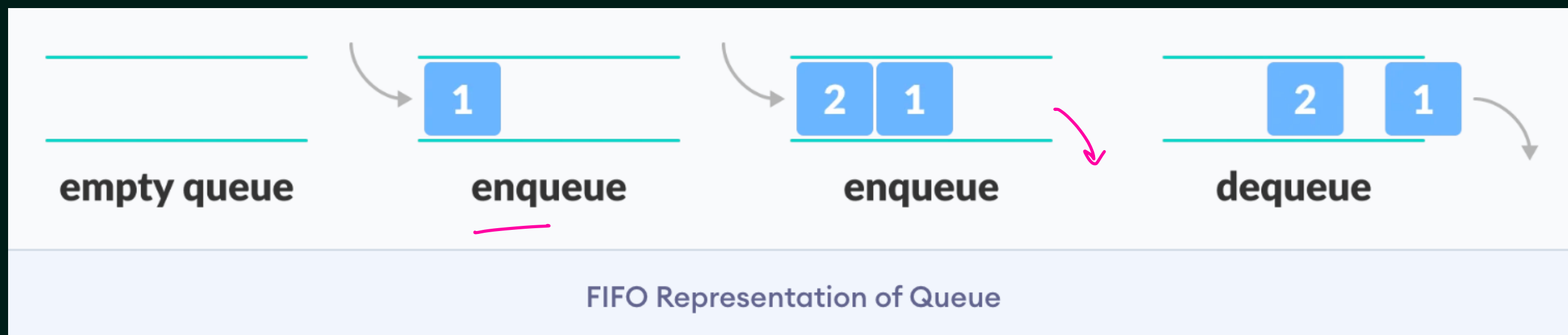
1. How Queue Works
2. Implementation of Queue
3. Implementation of Circular Queue

# How Queue Works

Queue follows the First In First Out (FIFO) rule - the item that goes in first is the item that comes out first.

In the above image, since 1 was kept in the queue before 2, it is the first to be removed from the queue as well. It follows the FIFO rule.

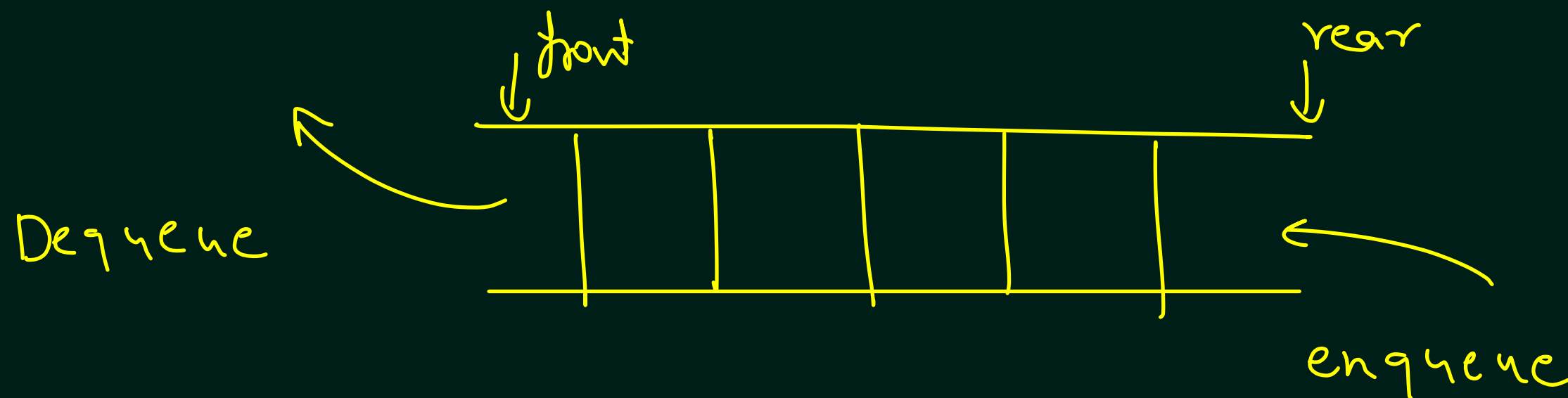
In programming terms, putting items in the queue is called enqueue, and removing items from the queue is called dequeue.



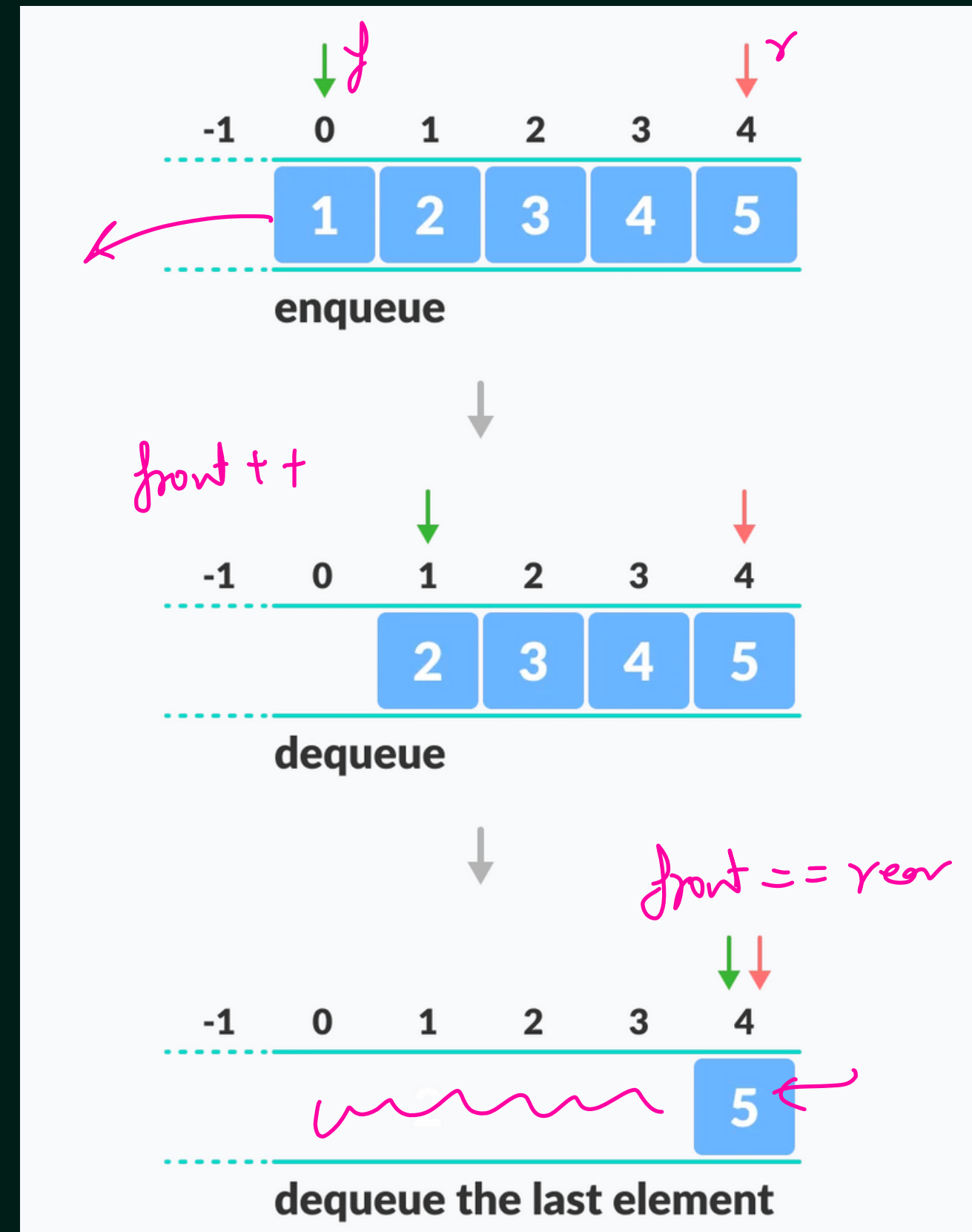
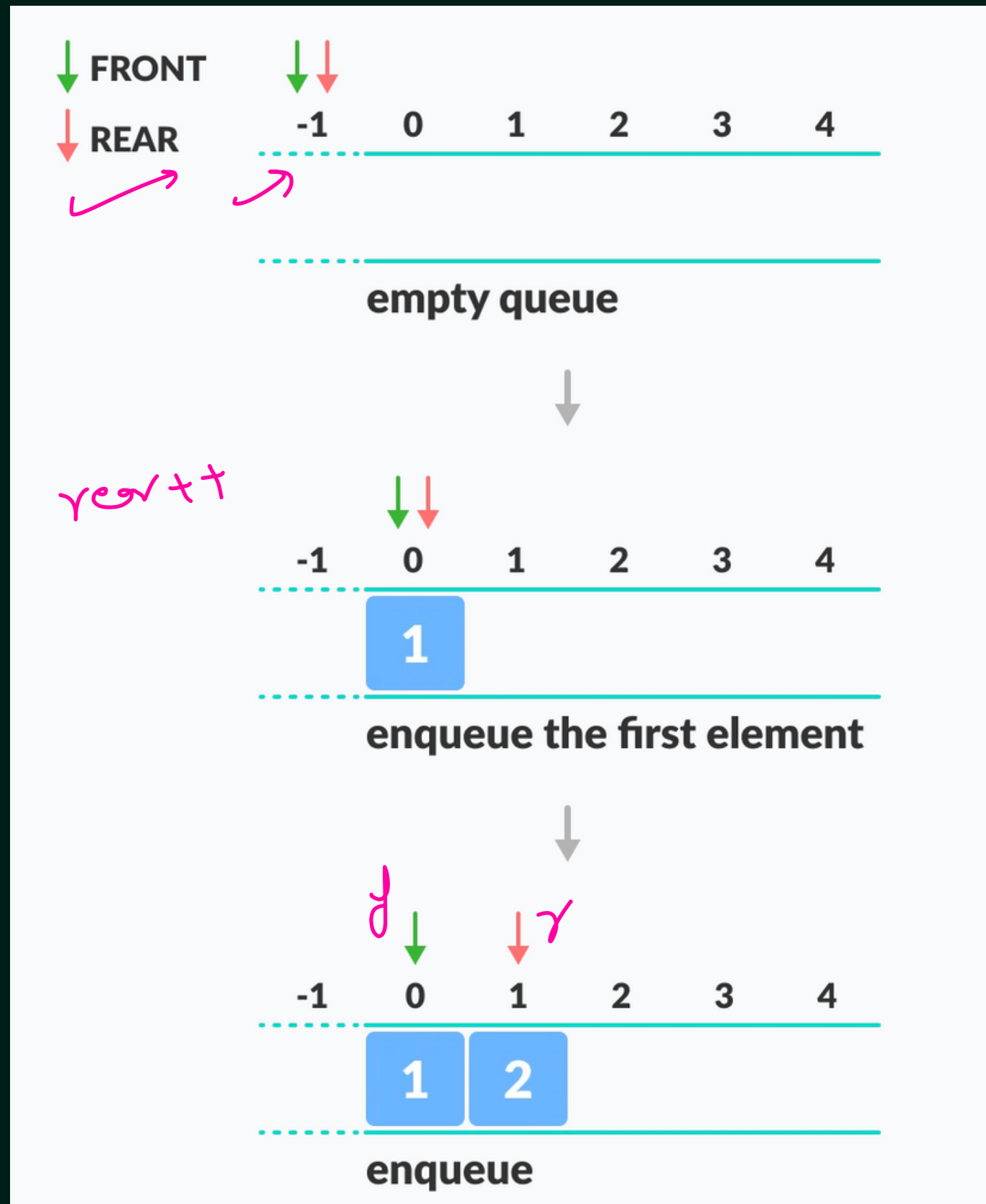
# Basic Operations of Queue

A queue is an object (an abstract data structure - ADT) that allows the following operations:

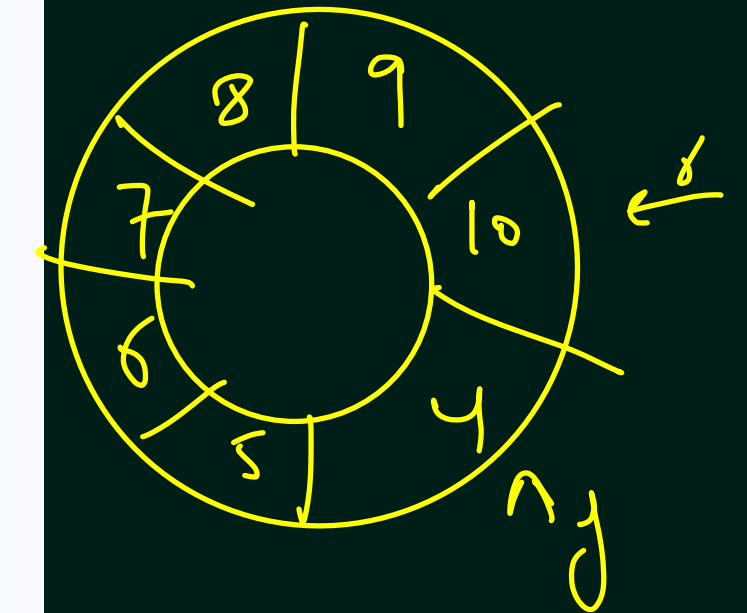
- Enqueue: Add an element to the end of the queue
- Dequeue: Remove an element from the front of the queue
- IsEmpty: Check if the queue is empty
- IsFull: Check if the queue is full
- Peek: Get the value of the front of the queue without removing it



# Implementation of Queue using Array



size = 5



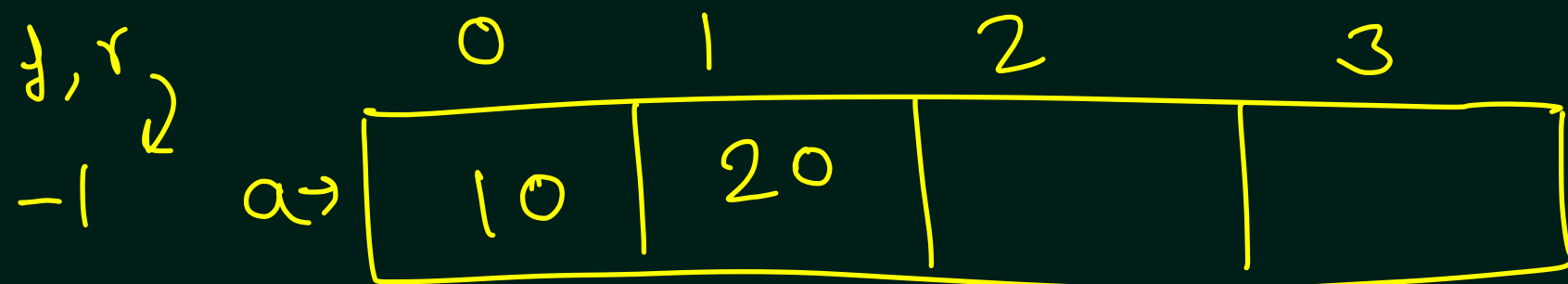


# Implementation of Queue using Array

```
public boolean enqueue(int data) {
    if(isFull()) {
        System.out.println("Queue is Full, Cannot add new elements");
        return false;
    }
    if(isEmpty()) {
        front = 0;
    }
    rear++;
    a[rear] = data;
    return true;
}
```

```
public int dequeue() {
    if(isEmpty()) {
        System.out.println("Queue is Empty, Cannot dequeue");
        return -1;
    }
    int temp = a[front];
    if(front == rear) {
        front = rear = -1;
        return temp;
    }
    front++;
    return temp;
}
```

size = 4

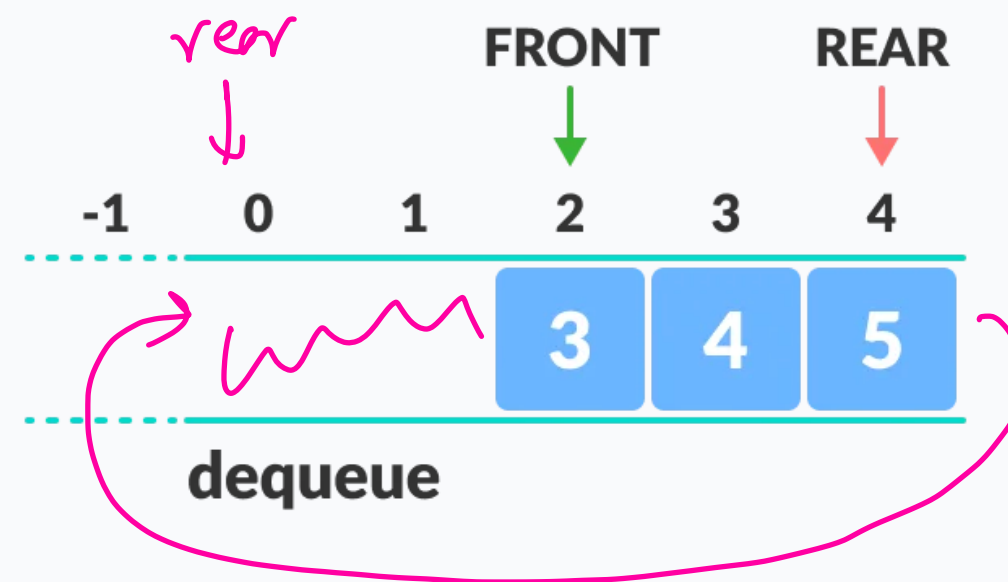


enqueue(20);  
dequeue()

kmp = 20

# Implementation of Circular Queue

The circular queue solves the major limitation of the normal queue. In a normal queue, after a bit of insertion and deletion, there will be non-usable empty space.

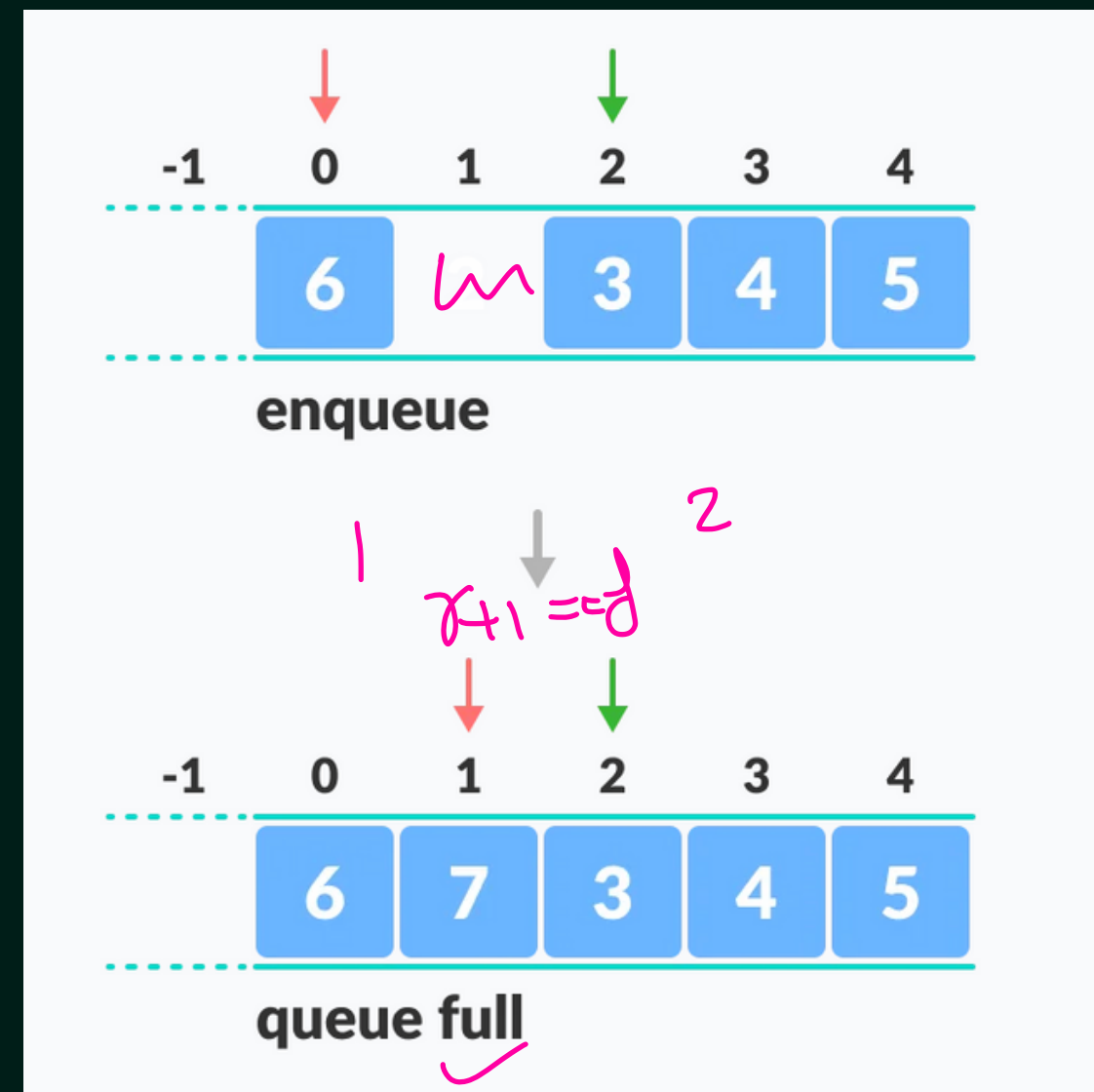
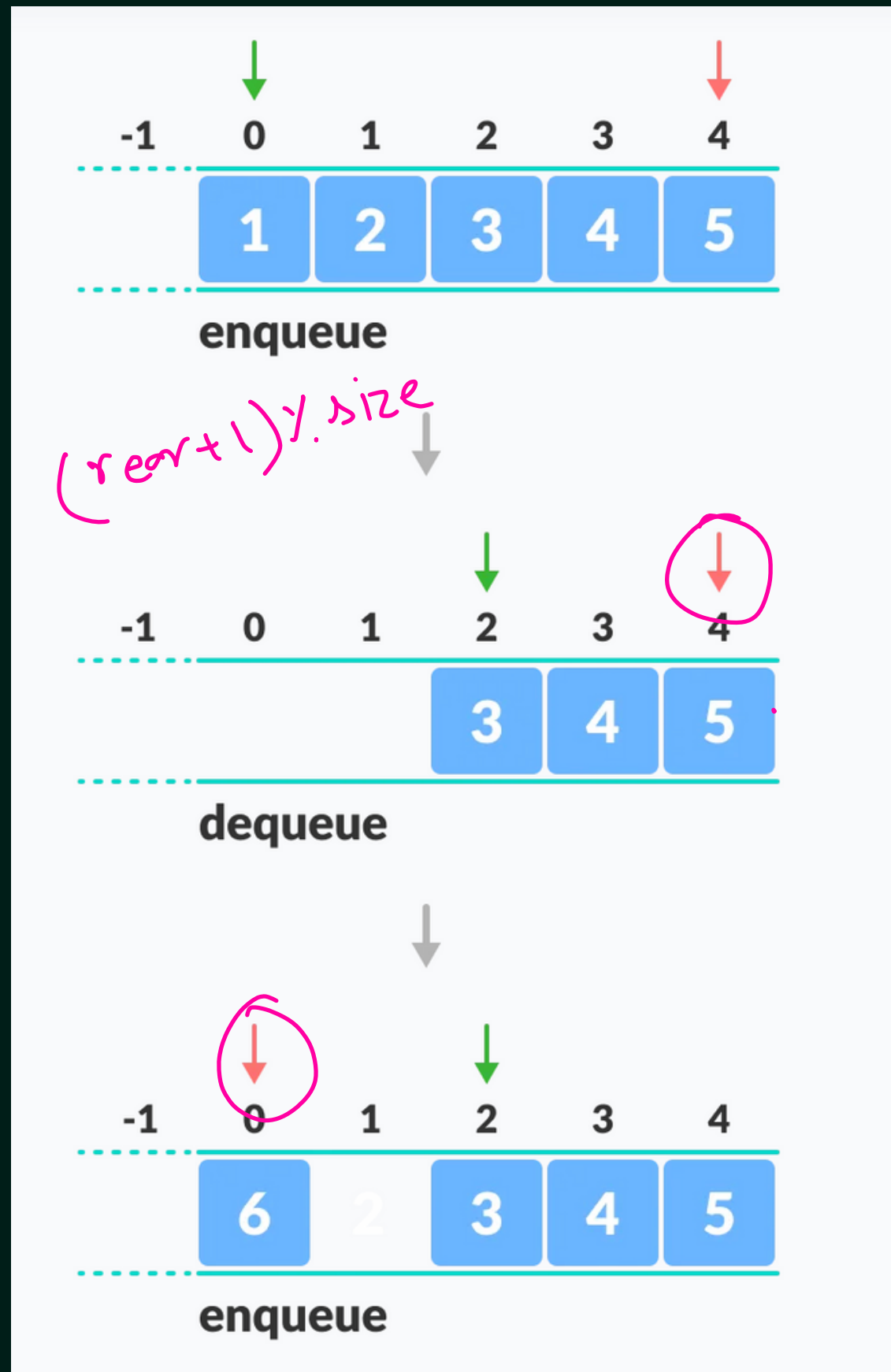


Limitation of the regular Queue

Circular Queue works by the process of circular increment i.e. when we try to increment the pointer and we reach the end of the queue, we start from the beginning of the queue.

# Implementation of Circular Queue

The circular queue solves the major limitation of the normal queue. In a normal queue, after a bit of insertion and deletion, there will be non-usuable empty space.

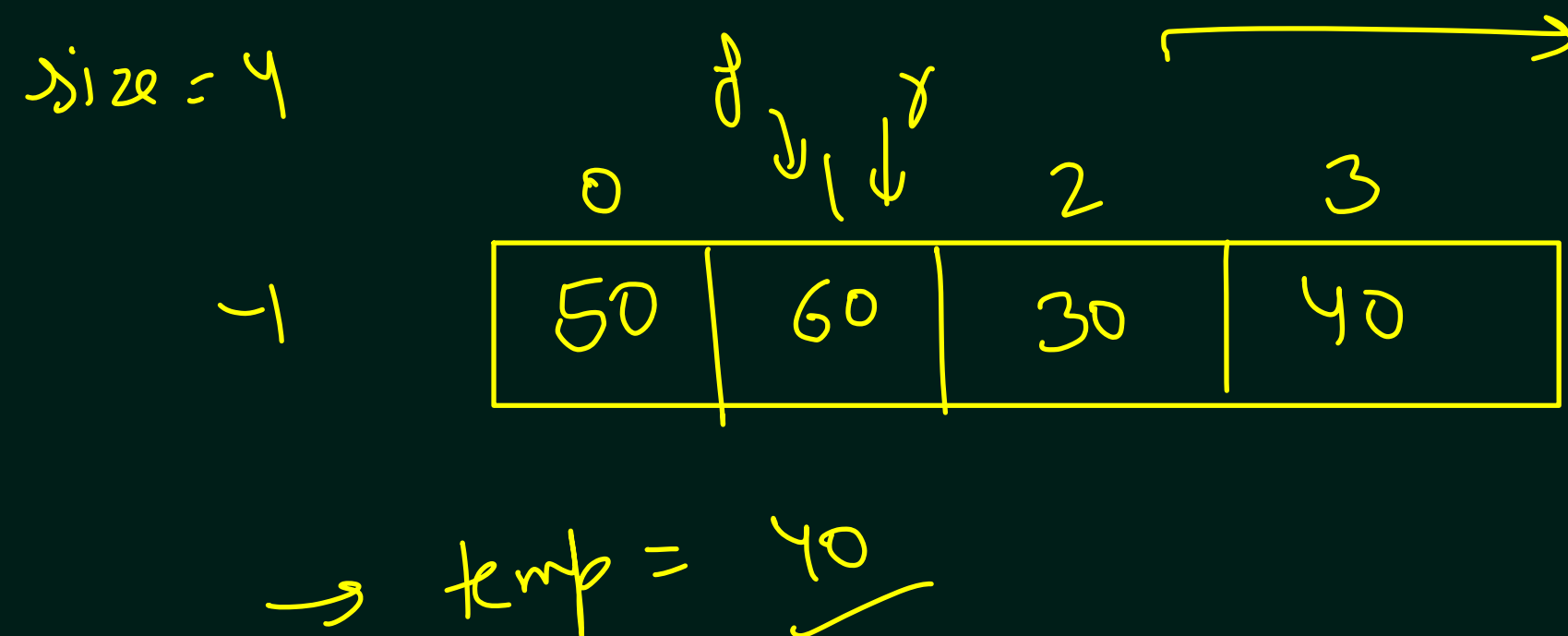




# Implementation of Circular Queue

```
public boolean enqueue(int data) {
    if(isFull()) {
        System.out.println("Queue is Full, Cannot add new elements");
        return false;
    }
    if(isEmpty()) {
        front = 0;
    }
    rear = (rear+1)%size;
    a[rear] = data;
    return true;
}
```

```
public int dequeue() {
    if(isEmpty()) {
        System.out.println("Queue is Empty, Cannot dequeue");
        return -1;
    }
    int temp = a[front];
    if(front == rear) {
        front = rear = -1;
        return temp;
    }
    front = (front+1)%size;
    return temp;
}
```



$$(3 + 1) \% 4$$

$$\downarrow$$

$$0$$

$$(0 + 1) \% 4$$